Toward Adaptive and Robust Machine Learning based Network Intrusion Detection Systems

Omar Anser

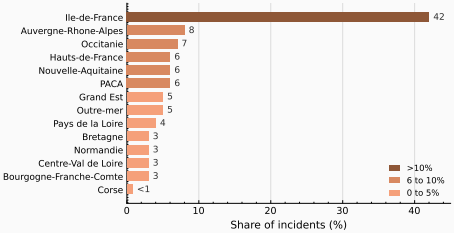
April 23, 2026

SnT — University of Luxembourg, Luxembourg

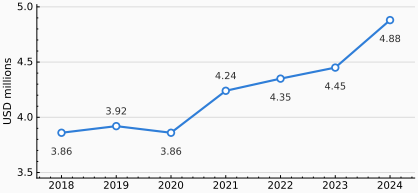
Introduction

Why this topic matters

- Modern societies and public services increasingly rely on computer networks $\longrightarrow$ Higher exposure to cyber incidents and service interruptions.
- ANSSI handled **4,386** security events in 2024 (**+15%** vs. 2023), including **1,361** confirmed incidents, i.e., nearly **4** per day.
- **$4.88M** average global cost of a data breach in 2024.
- **35%** of reported cyber incidents in the French health sector affected patient care.



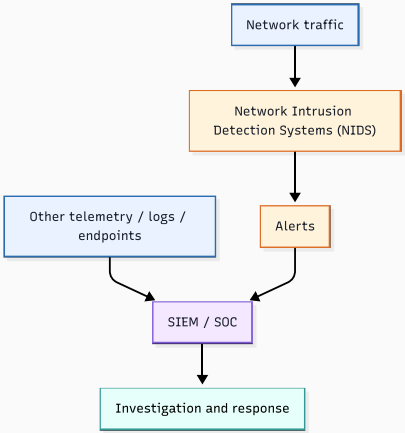
Source: **ANSSI, Panorama de la cybermenace 2024**



Source: **IBM Security, Cost of a Data Breach Report 2024**

Where NIDS fits in cyber defense

- Continuous monitoring to manage cyber risk
- SOCs and SIEM platforms centralize this monitoring
- Network traffic gives direct visibility into system communications
- NIDS turn traffic into alerts for investigation and response



⇒ **NIDS is one analytical layer inside the broader monitoring and response chain.**

Refs: NIST SP 800-86 (2006); Scarfone & Mell, Guide to Intrusion Detection and Prevention Systems (2007).

From signatures to ML-based NIDS

Rule-based NIDS

- Tools: **Snort**, **Suricata**, **Zeek**
- Detect only known attack patterns or protocol violations, leaving unknown ones undetected.
- Limited when payload content is encrypted

ML-based NIDS

- Example: **Random Forest (RF) on flows**
- Learn traffic patterns from historical data to generalize beyond exact rules.
- Can still detect suspicious behavior in encrypted traffic

Operational reality: mostly hybrid. For example, a SOC may use **Suricata/Zeek** for known alerts and **ML on flows** to flag suspicious HTTPS command-and-control behavior.

Refs: Snort [Roesch, 1999]; Zeek [Paxson, 1999]; Suricata (OISF); ML for NIDS [Sommer & Paxson, 2010]; encrypted traffic [Anderson & McGrew, 2016]; SANS Detection & Response Survey (2024).

ML-based NIDS as a prediction task

Network observation $\longrightarrow$ feature representation $\longrightarrow$ detection model $\longrightarrow$ prediction

Input and output

- A network observation can be a packet, a flow, a sequence, or a graph
- The model takes a numerical representation of this observation
- The output is either a traffic label or an anomaly score

Two common learning settings

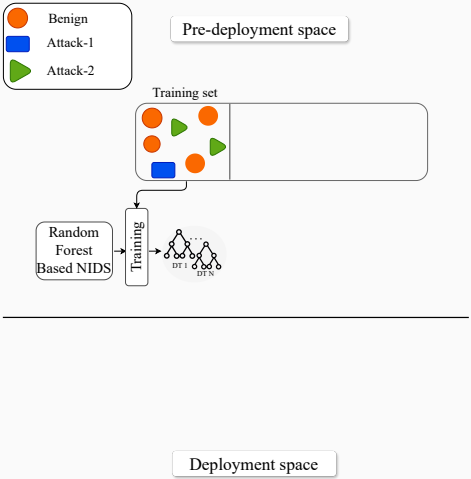
- **Classification**: trained with labelled benign and attack traffic
- **Anomaly detection**: trained mainly on benign traffic
- In anomaly detection, alerts are raised when the score exceeds a threshold

$\Rightarrow$ **This common ML setting leads to the standard train $\rightarrow$ validate $\rightarrow$ test $\rightarrow$ deploy workflow.**

Refs: Sommer & Paxson (2010); Buczak & Guven (2016).

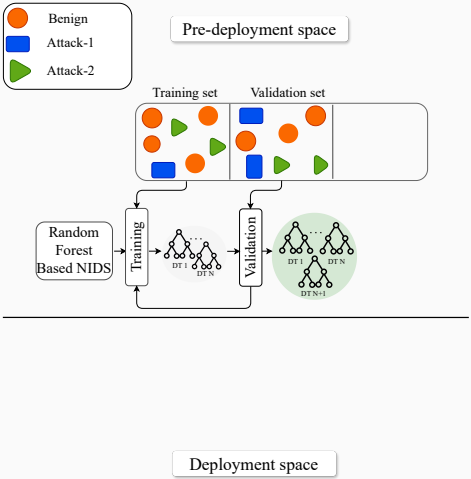
ML-based NIDS Workflow: Train → Validate → Test → Deploy

- The model is trained on a labelled **training** set $\mathcal{D}_{\text{train}}$.



ML-based NIDS Workflow: Train → Validate → Test → Deploy

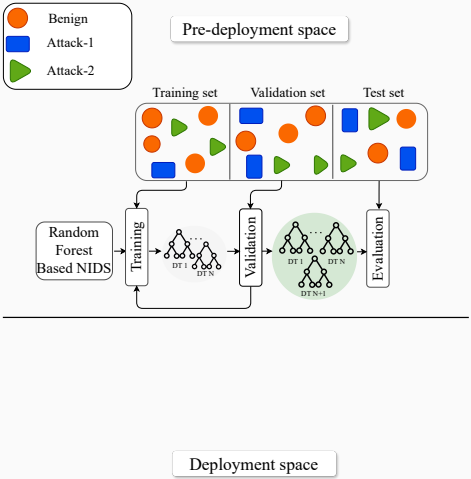
- The model is trained on a labelled **training** set $\mathcal{D}_{\text{train}}$.
- A separate **validation** split $\mathcal{D}_{\text{val}}$ is used to refine the model **configuration** (**hyperparameters**).



Hyperparameters: settings chosen before training that influence model behavior.
Example: In RF, too few trees can give unstable predictions, while too many can increase computation and overfit the training data.

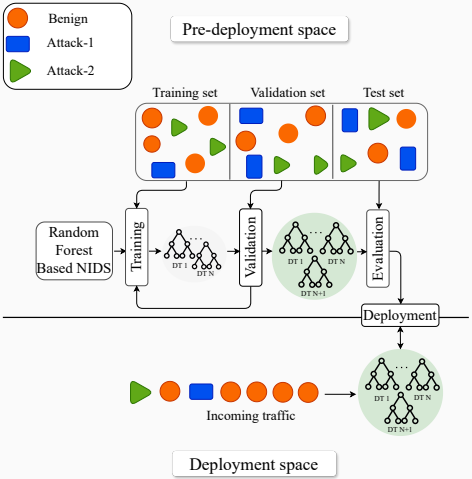
ML-based NIDS Workflow: Train → Validate → Test → Deploy

- The model is trained on a labelled **training** set $\mathcal{D}_{\text{train}}$.
- A separate **validation** split $\mathcal{D}_{\text{val}}$ is used to refine the model **configuration** (**hyperparameters**).
- A final **test** set, $\mathcal{D}_{\text{test}}$, measures performance on previously unseen data.



ML-based NIDS Workflow: Train → Validate → Test → Deploy

- The model is trained on a labelled **training** set $\mathcal{D}_{\text{train}}$.
- A separate **validation** split $\mathcal{D}_{\text{val}}$ is used to refine the model **configuration** (**hyperparameters**).
- A final **test** set, $\mathcal{D}_{\text{test}}$, measures performance on previously unseen data.
- The selected model is then **deployed** to monitor live traffic.



How ML has been used to improve NIDS

- Better detection performance $\longrightarrow$ BotMiner, Kitsune
  e.g., DDoS, botnet, web attacks
- Richer traffic representations $\longrightarrow$ Flow-MAE, NetVigil
  e.g., packet, flow, sequence, graph
- More practical deployment $\longrightarrow$ AOC-IDS, ENIDrift
  e.g., lightweight, online, real-time detection
- Better use of limited labels $\longrightarrow$ Active-SVDD, INSOMNIA
  e.g., active learning, semi-supervised learning, and pseudo-labeling
- Better robustness in practice $\longrightarrow$ CADE, OWAD
  e.g., adaptability to evolving traffic, resilience to adversarial attacks, and more challenging evaluation

Refs: BotMiner [Gu et al., 2008]; Kitsune [Mirsky et al., 2018]; Flow-MAE [Hang et al., 2023]; NetVigil [Hsieh et al., 2024]; AOC-IDS [Zhao et al., 2024]; ENIDrift [Wang, 2022]; Active-SVDD [Gornitz et al., 2009]; INSOMNIA [Pierazzi et al., 2021]; CADE [Yang et al., 2021]; OWAD [Hu et al., 2023].

How ML has been used to improve NIDS

- Better detection performance $\longrightarrow$ BotMiner, Kitsune
  e.g., DDoS, botnet, web attacks
- Richer traffic representations $\longrightarrow$ Flow-MAE, NetVigil
  e.g., packet, flow, sequence, graph
- More practical deployment $\longrightarrow$ AOC-IDS, ENIDrift
  e.g., lightweight, online, real-time detection
- Better use of limited labels $\longrightarrow$ Active-SVDD, INSOMNIA
  e.g., active learning, semi-supervised learning, and pseudo-labeling
- **Better robustness in practice $\longrightarrow$ CADE, OWAD**
  e.g., adaptability to evolving traffic, resilience to adversarial attacks, and more challenging evaluation

Refs: BotMiner [Gu et al., 2008]; Kitsune [Mirsky et al., 2018]; Flow-MAE [Hang et al., 2023]; NetVigil [Hsieh et al., 2024]; AOC-IDS [Zhao et al., 2024]; ENIDrift [Wang, 2022]; Active-SVDD [Gornitz et al., 2009]; INSOMNIA [Pierazzi et al., 2021]; CADE [Yang et al., 2021]; OWAD [Hu et al., 2023].

Focus of this thesis: robustness in ML-based NIDS

Robustness in this thesis

- Good performance on one fixed benchmark split is not sufficient to claim robustness.
- Before deployment, robustness depends on evaluation conditions that reveal detector weaknesses.
- After deployment, robustness means remaining effective when traffic and attack patterns evolve.

$\Rightarrow$ **ML-based NIDS robustness is studied both before deployment, through the evaluation process, and after deployment, through adaptability to evolving traffic.**

Roadmap of the presentation

Each part follows the same logic

State of the art → Limitations → Proposed contribution

Part I Evaluation

- Anser et al., *"TATA: Benchmark NIDS Test Sets Assessment and Targeted Augmentation,"* ESORICS 2025 (*main track*).

Part II Adaptation

- Anser et al., *"Auto-tuning of Hyper-parameters for Detecting Network Intrusions via Meta-learning,"* NOMS 2023 (*AnNet workshop*).
- Anser et al., *"Automated Machine Learning Configuration to Learn Intrusion Detectors on Attack-Free Datasets,"* LCN 2024 (*short paper*).
- Anser et al., *"An Empirical Study on Configuration Robustness of Unsupervised IDS,"* NOMS 2026 (*main track, to appear*).

Part 1 draft

High-level plan

1. Use the last figure of the workflow of ESORICS to position exactly where we are in the workflow and what we are targeting.
2. Use the rest of the slides of esorics it is perfect

Part 2 draft

High-level plan

1. Why ML-based NIDS still need explicit adaptation
2. State of the art: main adaptation families
3. Synthesis and gap analysis
4. Is configuration important? Evidence from our NOMS'26 study
5. How configuration is handled today: offline HPO
6. Why conventional HPO is not suited to fast re-adaptation
7. Research question (RQ1)
8. Contribution: meta-learning-based auto-configuration
9. Evaluation in supervised and anomaly-detection settings
10. Main takeaways of Part I

Questions?